

Computing $\|u\|_{L^2}$ from the Boundary

A primer on `lappy`'s boundary quadrature

`lappy` development notes

Abstract

This is a working introduction to the mathematics behind `lappy.eigfun.integrals`, aimed at someone who knows vector calculus and a little numerical analysis and wants to understand — and modify — the code as it stands today. It develops one identity, shows why discretizing it is harder than it looks, and explains each of the three things the code has to get right before it can claim an answer to 10^{-13} . Everything is checkable: every number quoted can be reproduced by the snippets in the text, and the exercises at the end are the ones worth doing first.

The companion document `corner_quadrature.pdf` is the research record — it derives the corner rule properly and reports what was tried and discarded. Read this one first.

Contents

1	The problem, and why the boundary	2
2	The identity	2
2.1	Statement	2
2.2	Proof, in full	3
2.3	Three consequences worth internalizing	3
2.4	A worked example you should reproduce	4
3	Discretizing: panels and nodes	4
4	Why this is not just “call Gauss–Legendre”	5
4.1	What an eigenfunction looks like at a corner	5
4.2	Gauss–Legendre on r^γ	5
4.3	The criterion <code>lappy</code> uses, and the one it used to use	6
5	The corner-adapted rule, in one page	6
5.1	Two things that look like bugs	7
6	Three requirements, not one	7
7	What is out of reach, and how the code says so	8
8	How to check your own work	9
9	Scope: what this machinery is <i>not</i> for	9

1 The problem, and why the boundary

We solve $-\Delta u = \lambda u$ in a planar domain Ω with $u = 0$ on $\partial\Omega$. A method-of-particular-solutions (MPS) eigensolver gives us λ and a coefficient vector: u is a finite combination of functions that each solve the PDE *exactly* in Ω , chosen so that u is as close to zero on $\partial\Omega$ as the basis allows. So u is not an approximation in the finite-element sense; it is an exact solution of the PDE with a small nonzero boundary trace.

Very often we then need the L^2 norm,

$$\|u\|_{L^2(\Omega)}^2 = \int_{\Omega} u^2 \, dA, \quad (1)$$

because eigenfunctions are only defined up to scale and nearly every downstream formula wants them normalized:

- to return orthonormal eigenfunctions from a degenerate cluster;
- to certify an eigenvalue — the Moler–Payne bound is a ratio of a boundary sup-norm to an interior L^2 norm;
- to evaluate shape derivatives (Hadamard-type formulas), which are boundary integrals of normalized eigenfunctions.

The obvious route is a cubature rule on the interior: triangulate Ω , sum $w_i u(x_i)^2$. This works, and `lappy` still has it (`lappy.cubature`), but it is the expensive and fragile half of any computation that uses it. A triangulation is a data structure to build, refine and store; on a curved boundary the polygonal mesh does not even cover Ω exactly; on a spiral domain the mesh takes longer to build than the eigenvalue took to find.

The alternative is to compute an *area* integral from *boundary* data only. That is what the Rellich identity does, and it is the subject of this primer.

2 The identity

2.1 Statement

Fix any point $x_0 \in \mathbb{R}^2$ — it is a free parameter, and nothing below assumes anything about it. Write $r := x - x_0$ and let n be the outward unit normal.

Proposition 1 (Rellich–Pohozaev, Dirichlet form). *Let u solve $-\Delta u = \lambda u$ in $\Omega \subset \mathbb{R}^2$ with $u = 0$ on $\partial\Omega$. Then*

$$\int_{\Omega} u^2 \, dA = \frac{1}{2\lambda} \int_{\partial\Omega} (r \cdot n) \left(\frac{\partial u}{\partial n} \right)^2 \, ds. \quad (2)$$

Everything on the right is boundary data: the geometry $(r \cdot n)$ and the normal derivative of u . No interior evaluation appears at all.

2.2 Proof, in full

The trick is to multiply the equation by $r \cdot \nabla u$ — the derivative of u in the radial direction from x_0 — and integrate. This is the standard Pohozaev manoeuvre and it is worth doing once by hand.

Proof. Both sides of $-\Delta u = \lambda u$ are multiplied by $r \cdot \nabla u$ and integrated over Ω . Take the right-hand side first. Since $u(r \cdot \nabla u) = \frac{1}{2} r \cdot \nabla(u^2)$, the divergence theorem gives

$$\lambda \int_{\Omega} u(r \cdot \nabla u) = \frac{\lambda}{2} \int_{\Omega} r \cdot \nabla(u^2) = \frac{\lambda}{2} \left[\int_{\partial\Omega} (r \cdot n) u^2 - \int_{\Omega} u^2 \nabla \cdot r \right] = -\lambda \int_{\Omega} u^2, \quad (3)$$

using $u = 0$ on $\partial\Omega$ to kill the boundary term and $\nabla \cdot r = 2$ in the plane. (In $\mathbb{R}^d$ that 2 would be d ; the identity's constants are dimension-dependent, and `lappy` is planar throughout.)

For the left-hand side, integrate by parts once:

$$\int_{\Omega} \Delta u (r \cdot \nabla u) = \int_{\partial\Omega} \frac{\partial u}{\partial n} (r \cdot \nabla u) - \int_{\Omega} \nabla u \cdot \nabla (r \cdot \nabla u). \quad (4)$$

For the last integrand, note $\nabla(r \cdot \nabla u) = \nabla u + (r \cdot \nabla) \nabla u$, so

$$\nabla u \cdot \nabla (r \cdot \nabla u) = |\nabla u|^2 + \frac{1}{2} r \cdot \nabla (|\nabla u|^2), \quad (5)$$

and applying the divergence theorem to the second piece,

$$\int_{\Omega} \nabla u \cdot \nabla (r \cdot \nabla u) = \int_{\Omega} |\nabla u|^2 + \frac{1}{2} \int_{\partial\Omega} (r \cdot n) |\nabla u|^2 - \int_{\Omega} |\nabla u|^2 = \frac{1}{2} \int_{\partial\Omega} (r \cdot n) |\nabla u|^2. \quad (6)$$

Now use the boundary condition. Because $u \equiv 0$ along $\partial\Omega$, its tangential derivative vanishes there, so the gradient is purely normal: $\nabla u = (\partial u / \partial n) n$ on $\partial\Omega$. Hence $r \cdot \nabla u = (r \cdot n) \partial u / \partial n$ and $|\nabla u|^2 = (\partial u / \partial n)^2$, and (4) collapses to

$$\int_{\Omega} \Delta u (r \cdot \nabla u) = \int_{\partial\Omega} (r \cdot n) \left(\frac{\partial u}{\partial n} \right)^2 - \frac{1}{2} \int_{\partial\Omega} (r \cdot n) \left(\frac{\partial u}{\partial n} \right)^2 = \frac{1}{2} \int_{\partial\Omega} (r \cdot n) \left(\frac{\partial u}{\partial n} \right)^2. \quad (7)$$

Since $\int (-\Delta u)(r \cdot \nabla u) = \lambda \int u(r \cdot \nabla u)$, the two computations give $-\frac{1}{2} \int_{\partial\Omega} (r \cdot n) (\partial u / \partial n)^2 = -\lambda \int_{\Omega} u^2$, which is (2). $\square$

2.3 Three consequences worth internalizing

(i) **x_0 is free, and that is a gift.** The proof never used any property of x_0 . So (2) holds for every choice, and if you evaluate the right-hand side numerically at two different x_0 and get two different answers, the difference is *error* — you have a diagnostic that costs nothing and needs no reference solution. `lappy` uses this, with a caveat we return to in §8.

(ii) **A corner can be made to disappear.** If x_0 sits exactly at a corner, then on both straight edges meeting there, $r = x - x_0$ points *along* the edge while n is perpendicular to it, so $r \cdot n \equiv 0$ on those edges. Whatever nasty behaviour $\partial u / \partial n$ has at that corner is multiplied by an exact zero. This is why `default_x0` places x_0 on a singular corner when there is one. It handles one corner exactly and for free — but x_0 is a single point, so a domain with several bad corners cannot be rescued this way, which is the entire reason the rest of this document exists.

(iii) **Setting $u \equiv 1$ is not allowed, but the same trick is.** You cannot test (2) with $u = 1$ (it is not an eigenfunction). But the divergence theorem alone gives a closely related identity that is pure geometry,

$$\int_{\partial\Omega} (r \cdot n) \, ds = \int_{\Omega} \nabla \cdot r \, dA = 2 |\Omega|, \quad (8)$$

again for *any* x_0 . Equation (8) contains no λ , no eigenfunction and no basis, so it isolates the quadrature’s treatment of the *geometry*. It turns out to be the sharpest single test in the whole subject; §6 is about what it revealed.

2.4 A worked example you should reproduce

Take the unit disk, $\Omega = \{|x| < 1\}$, and $x_0 = 0$, so $r \cdot n = 1$ everywhere on $\partial\Omega$. The Dirichlet eigenfunctions are $u = J_m(kr) \cos(m\theta)$ with $k = j_{m,n}$, the n -th positive zero of J_m . On the boundary $\partial u / \partial n = k J'_m(k) \cos(m\theta)$, so the right side of (2) is

$$\frac{1}{2k^2} \int_0^{2\pi} k^2 J'_m(k)^2 \cos^2(m\theta) \, d\theta = \frac{\pi}{2} J'_m(j_{m,n})^2 = \frac{\pi}{2} J_{m+1}(j_{m,n})^2, \quad (9)$$

using $J'_m(j_{m,n}) = -J_{m+1}(j_{m,n})$, and that is exactly the classical formula for $\|u\|^2$ on the disk. The identity is not merely true, it is *recognizably* true on the one domain where you know everything.

3 Discretizing: panels and nodes

To evaluate (2) we need a quadrature rule on $\partial\Omega$: nodes x_i , weights w_i , normals n_i , and $\int f \, ds \approx \sum_i w_i f(x_i)$.

`lappy` builds this once per solve and reuses it for every λ , because the node set depends only on the geometry, on an upper bound $\lambda_{\max}$ for the eigenvalues of interest, and on a requested accuracy — never on the basis or on u . That is the design constraint to keep in mind: a routine that needs to look at u to place nodes cannot live here.

The boundary is a list of *segments* (a line, an arc, a spline), each parametrized by normalized arclength $\tau \in [0, 1]$, so that a node at τ sits at arclength $\tau \cdot L_{\text{seg}}$ and the weights carry no per-node Jacobian. Each segment is cut into *panels*, and each panel gets its own rule:

<code>legendre</code>	plain Gauss–Legendre; used on smooth stretches
<code>cornerjac</code> , <code>cornerinterp</code>	corner-adapted rules, anchored at a corner

The entry point is

```
from lappy.eigfun_integrals import boundary_quadrature
bq = boundary_quadrature(domain, lam_max, precision=1e-13)
```

and `bq` carries `pts`, `normals`, `tangents`, `wts`, its own reference point `x0`, the `panels` it was built from, and two fields that report honesty rather than data: `precision` and `shortfalls` (§7).

Given `bq`, evaluating the identity is three lines:

```
from lappy.eigfun_integrals import eigfun_cauchy_data, gram
ed = eigfun_cauchy_data(basis, lam, coef, bq)    # u, du/dn, du/dt at nodes
G = gram(ed, lam, bq)                            # (mult x mult) Gram matrix
```

For a single eigenfunction $\mathbf{G}$ is 1×1 and equals $\|u\|^2$. For a degenerate cluster of multiplicity m it is the $m \times m$ matrix of inner products $\langle u_i, u_j \rangle$, from which `lowdin_transform` produces an orthonormal combination. Note the order of operations: the basis is contracted with the coefficients *first*, and only then are integrals formed. Never assemble a basis-level Gram matrix and sandwich it — §9.

4 Why this is not just “call Gauss–Legendre”

If $\partial\Omega$ is smooth and u is smooth, Gauss–Legendre converges spectrally and there is no story. The story is corners.

4.1 What an eigenfunction looks like at a corner

Put a corner at the origin with interior angle α , edges along $\theta = 0$ and $\theta = \alpha$, and define

$$\nu := \frac{\pi}{\alpha}. \quad (10)$$

So a right angle has $\nu = 2$, a straight-through “corner” $\nu = 1$, a 270° reentrant corner $\nu = 2/3$, and a slit ($\alpha \rightarrow 2\pi$) $\nu \rightarrow 1/2$. Every Dirichlet solution has the local expansion

$$u(r, \theta) = \sum_{k \geq 1} c_k J_{k\nu}(\sqrt{\lambda} r) \sin(k\nu\theta), \quad (11)$$

which is worth reading twice: the angular functions $\sin(k\nu\theta)$ are exactly those vanishing on both edges, and the radial factors are then forced to be Bessel functions of matching order.

On the edge $\theta = 0$ the normal derivative is $-\frac{1}{r}\partial_\theta u$, and using $J_\mu(z) = (z/2)^\mu A_\mu(z^2)$ with A_μ entire and $A_\mu(0) \neq 0$,

$$\left. \frac{\partial u}{\partial n} \right|_{\theta=0} = r^{\nu-1} F(r), \quad \text{so} \quad \left(\frac{\partial u}{\partial n} \right)^2 = r^{2\nu-2} G(r), \quad (12)$$

where F and G are series in powers $r^{k\nu}$ and r^{2q} . **This is the single most important formula in this document.** The integrand of (2) near a corner is a smooth series multiplied by $r^{2\nu-2}$.

4.2 Gauss–Legendre on r^γ

So the question is: how well does a Gauss rule integrate τ^γ with $\gamma = 2\nu - 2$? Write it down and try it:

```
from lappy.quad import smooth_power_error
smooth_power_error(gamma, order) # relative error on int_0^1 tau^gamma
```

$\gamma = 2\nu - 2$	ν	α	$n = 16$	$n = 64$	fitted rate
1/3	7/6	154°	8.0×10^{-5}	2.1×10^{-6}	$n^{-2.6}$
2/3	4/3	135°	1.2×10^{-5}	1.2×10^{-7}	$n^{-3.3}$
1	3/2	120°	0	0	exact
3/2	7/4	103°	1.7×10^{-7}	1.8×10^{-10}	$n^{-5.0}$
11.55	6.78	27°	8.7×10^{-16}	3.5×10^{-16}	exact

Three things to take from this table.

Integer γ is exact. An n -point Gauss rule integrates polynomials of degree $\leq 2n - 1$ exactly, and τ^γ with integer γ is a polynomial. So $\nu \in \{1, \frac{3}{2}, 2, \frac{5}{2}, 3, \dots\}$ — which includes every right angle and every 120° corner — costs nothing at all.

Non-integer γ converges algebraically, at rate $n^{-(2\gamma+2)}$. The function τ^γ is continuous but not analytic at 0: some derivative blows up. Gauss–Legendre’s spectral convergence needs analyticity, and without it the classical result is an algebraic rate governed by how many derivatives exist. The fitted rates above match $2\gamma + 2$ to two digits (2.64 vs 2.67, 3.31 vs 3.33, 4.96 vs 5.00), which is a satisfying check to run yourself.

The rate is what matters, not the exponent’s sign. At $\gamma = 11.55$ the predicted rate is n^{-25} : the function is so smooth that Gauss is at roundoff by order 16, even though γ is irrational. At $\gamma = 2/3$ the rate is $n^{-3.3}$ and you will *never* reach 10^{-13} — order 64 gives 10^{-7} , and quadrupling again buys three digits.

4.3 The criterion lappy uses, and the one it used to use

Here is the practical payoff, and a mistake worth learning from. For a long time the code decided a corner “needs the special rule” when $\nu < 1$, i.e. when the corner is *reentrant*, on the reasoning that a convex corner has bounded u and bounded $\partial u / \partial n$, so a smooth rule should be fine.

That reasoning is wrong, and the table above says why. At a 135° corner, $\nu = 4/3$: the eigenfunction is bounded, its normal derivative $r^{1/3}$ even *vanishes* at the corner — and the integrand is $r^{2/3}$, which Gauss integrates at rate $n^{-3.3}$. Nothing is singular in the everyday sense, and the quadrature still fails. What matters is not whether $2\nu - 2$ is negative but whether it is an *integer*.

Equally, the opposite over-correction is also wrong: “use the corner rule whenever ν is not an integer” hands the special rule to $\nu = 6.78$ corners that do not need it and makes them *worse*, because the corner rule is built to be exact on a singular class and is not even exact on constants (§5.1). So lappy asks the question directly — what does a smooth rule achieve on $\tau^{2\nu-2}$ at the order this segment is getting? — and uses the corner rule exactly when the answer is not good enough. Measured on real domains, by refining the node set and differencing the integral:

domain	nodes	before	after
right_trapezoid	84 $\rightarrow$ 212	1.8×10^{-6}	2.9×10^{-16}
GW1	204 $\rightarrow$ 440	3.2×10^{-5}	5.0×10^{-13}
iso_tri_h05	96 $\rightarrow$ 224	1.2×10^{-11}	4.8×10^{-16}

5 The corner-adapted rule, in one page

When a corner does need help, the fix is to build a rule matched to the class (12) rather than to polynomials. Two ideas, both standard:

Absorb the singularity into the weight. Gauss–Jacobi rules integrate $\int_0^1 \tau^\gamma (\text{polynomial}) d\tau$ exactly, with γ as a weight exponent. Taking $\gamma = 2\nu - 2$ handles the $r^{2\nu-2}$ prefactor by construction, leaving a smooth series for the polynomial part.

Rationalize the exponents. G in (12) contains powers $r^{k\nu}$ and r^{2q} — two incommensurate families. A substitution $\tau = t^q$ with $\nu = p/q$ turns both into integer powers of t , so a polynomial rule in t is exact on the whole class. This works beautifully in theory and fails in practice for $q \geq 4$: the innermost node lands at $t_{\min}^q$, which underflows the coordinate representation (a node at $p_0 + \tau L \hat{d}$ rounds onto p_0 itself), and the accuracy is destroyed. `lappy` therefore takes the node *placement* from the mild exponent ν and gets exactness from the *weights* instead, via an interpolatory rule on the true exponent set. That is the difference between `cornerjac` (substitution; only safe for straight edges with $2/\nu$ integral) and `cornerinterp` (the general case).

Two practical consequences follow, and both look like bugs the first time.

5.1 Two things that look like bugs

The weights do not sum to the panel length. A corner rule is exact on $\{\gamma + k\nu + m\}$, a set that does not contain τ^0 . It therefore does not integrate constants, and $\sum_i w_i \neq L$ on that panel. Renormalizing would destroy exactness on the class the rule exists for. (This is also why feeding a corner rule an integrand with no corner structure — a plane wave, say — gives a bad answer, and why a naive “always use the corner rule” criterion loses.)

Accuracy is not monotone in the order. Past a ν -dependent threshold, raising the order makes things *worse*: the integrand’s dynamic range ($\tau^{2\nu-2} \sim 6 \times 10^8$ at $\tau \sim 10^{-9}$) against correspondingly tiny weights means roundoff, not truncation, dominates. Order selection therefore *scans* a calibrated curve for the best order; it never increases the order until a target is met.

6 Three requirements, not one

Choosing each panel’s order is where the code spends its cleverness. A panel must resolve all of the following, and the required order is the largest of the three.

1. Oscillation. An eigenfunction oscillates at wavenumber $\sqrt{\lambda}$, and the integrand is a *product* of two derivatives, so it oscillates at $2\sqrt{\lambda}$. Over a segment of length L that is a total phase $k = 2\sqrt{\lambda_{\max}} L$, and `smooth_order_for_precision` finds the smallest Gauss order integrating $e^{ik\tau}$ to the target. Model integrand, closed-form answer, so the sizing is self-certifying rather than calibrated offline.

2. The corner class. As §4: if the smooth rule cannot handle $\tau^{2\nu-2}$, the panel is corner-anchored and its order comes from a model integrand in the corner’s own exponent family (`corner_order_for_precision`).

3. The parametrization. This one is easy to forget, and it is the subject of the newest part of the code. Panels live in *normalized arclength*. For a straight edge or a circular arc, that parametrization is exact and free. For a curve of varying speed — an ellipse — the arclength map is analytic but its strip of analyticity narrows as the curve becomes eccentric, and Gauss–Legendre’s convergence rate narrows with it.

You can see this with no eigenfunction at all, using the pure-geometry identity (8):

order	16	32	64	128	192	256
disk	3e-16	3e-16	3e-16	3e-16	3e-16	3e-16
ellipse $a = 2$	5e-03	2e-04	4e-07	3e-12	5e-15	2e-15
ellipse $a = 4$	4e-02	1e-02	2e-03	8e-05	3e-06	1e-07
ellipse $a = 6$	6e-02	3e-02	1e-02	2e-03	5e-04	1e-04

A circle is exact at every order because its arclength map is linear. An $a = 2$ ellipse needs about 170 nodes where the oscillation model asked for 46. An $a = 6$ ellipse cannot be resolved at any order you would want to use. `geometry_order_for_precision` sizes each segment on (8), so this is caught automatically — and costs nothing on every exactly-parametrized boundary, because there the geometry order never exceeds the oscillation order.

Remark 1. A natural guess is that the culprit is the interpolation tolerance of the arclength table. It is not: the error is identical to three digits at $\text{tol} = 10^{-4}$, 10^{-6} and 10^{-8} , while the table’s build cost goes 0.0s, 0.9s, 65s. Refining it is pure expense. The lesson is worth generalizing — before optimizing a suspected cause, vary it and check that the symptom responds.

7 What is out of reach, and how the code says so

Some geometry is genuinely hard, and the design goal is not to pretend otherwise. Three examples, all in the benchmark suite:

- a **near-slit corner** ($\nu \rightarrow \frac{1}{2}$), where the integrand $r^{2\nu-2} \rightarrow r^{-1}$ is barely integrable and 90% of the integral lives within 10^{-9} of the corner;
- an **eccentric ellipse**, per §6;
- a **coiled spiral**, with many corners and long thin channels.

For these, `boundary_quadrature` does three things instead of quietly returning a wrong number:

1. `bq.precision` reports what the rule can support, not what was asked for. A singular corner that had to be demoted to a smooth rule gets ∞ , because there is no bound at all in that case.
2. `bq.shortfalls` names each corner or segment that fell short, and why.
3. a warning distinguishes the *cause* — in particular an eccentric ellipse warns that the *parametrization* is the limit, so you do not go hunting for a bug in the eigenfunction.

There is a fourth class, currently labelled rather than solved: a stadium has exact parametrization and integer ν , yet its integrals sit near 10^{-4} , because its boundary data carries far more harmonics than $2\sqrt{\lambda}L$ predicts. No geometry-only rule can anticipate that — it is a property of the solution, not of the domain — so the honest tool there is the a posteriori check of the next section.

8 How to check your own work

Refine and difference (the reliable one).

```
from lappy.eigfun_integrals import verify_gram
G, G_ref, err = verify_gram(basis, lam, coef, bq, domain)
```

recomputes the Gram on a systematically refined node set and returns the change. This measures the quadrature and nothing else. It is the instrument to reach for.

Vary x_0 (the free detector, with a caveat). By Proposition 1 the answer cannot depend on x_0 , so evaluating `gram(ed, lam, bq, x0=...)` at several points and looking at the spread costs three matrix products and needs no reference. It is what first exposed the $\nu < 1$ mistake. But it is a detector, not an error bar: it also carries the *eigenvalue*'s error, since λ enters through $1/(2\lambda)$ and through the Cauchy data, so an inexact λ makes u not quite an eigenfunction and the x_0 -dependence reappears. Measured, the two can differ by three orders in either direction — on one domain the spread reads 3.3×10^{-12} where the quadrature error is 1.4×10^{-13} ; on another it reads 4.1×10^{-11} where the quadrature error is 1.0×10^{-8} .

Compare against closed forms (the ground truth). `lappy.reference` has exact eigenfunctions and exact L^2 norms for rectangles, isosceles right triangles, disks, circular sectors of arbitrary angle, and polyominoes. These are the only unimpeachable references, and every test in `tests/test_eigfun_integrals.py` uses one.

Remark 2 (on instruments). Do not use `mpmath.quad` as a reference for these integrands. As $\nu \rightarrow \frac{1}{2}$ the endpoint behaviour approaches r^{-1} and it errs by 4×10^{-2} even at 40 digits — manufacturing convincing, entirely spurious, convergence plateaus. This cost a real amount of time.

9 Scope: what this machinery is *not* for

The rule is matched to *eigenfunction*-level data. At a corner, an exact eigenfunction's local structure is purely the corner family (11), which is exactly what the rule integrates.

It is not appropriate for a basis-level $N \times N$ Gram matrix. A basis column centred at some other corner is plain analytic at this one, with $O(1)$ amplitude at every corner simultaneously — the wrong class entirely. Measured, the corner rule beats a graded rule by 3–8 orders on corner-family blocks and *loses* by 2–4 on mixed ones, which is why `lappy` retired the basis-level path rather than adapting it. Hence the slogan in the module docstring: evaluate first, sandwich never. Contract the basis with the coefficients, *then* integrate.

10 Exercises

These are in the order I would do them. Each should take minutes, and each has a number you can check against.

Exercise 1 (the geometry identity). For `disk(1.0)`, `rect(1,1)` and `ellipse(2,1)`, build a node set and evaluate $\sum_i w_i (x_i - x_0) \cdot n_i$ against $2|\Omega|$, for two or three different x_0 . Which domains are exact, and which is not? Then re-run the ellipse with `resolve_geometry=False` and explain the difference.

Exercise 2 (the disk, in closed form). Using `lappy.reference.disk_eigfun`, form the Cauchy data of $J_2(j_{2,1}r) \cos 2\theta$ on the boundary by hand, evaluate (2) with the quadrature, and compare to $\frac{\pi}{2} J_3(j_{2,1})^2$. You should get 10^{-15} or better. Now do it for a circular sector of angle $\frac{3}{2}\pi$ with `sector_eigfun`, whose apex is a reentrant corner, and try x_0 at the apex versus x_0 elsewhere.

Exercise 3 (the rate law). Reproduce the τ^γ table with `smooth_power_error` and fit the convergence rate for $\gamma = 1/3, 2/3, 3/2$. Confirm $n^{-(2\gamma+2)}$. Then try $\gamma = 4.5$ and explain why the fit breaks down. (Hint: look at the size of the numbers, not the trend.)

Exercise 4 (the criterion). Build a regular octagon ($\nu = 4/3$ at every corner) and compare `boundary_quadrature(..., nonintegral=False)` with the default. Inspect `bq.panels` to see which rules were assigned, then use `verify_gram` on a real MPS eigenfunction to measure what changed. Predict the answer for a regular hexagon ($\nu = 3/2$) *before* running it.

Exercise 5 (reading a refusal). Build a node set for `ellipse(6,1)` and for a sector of angle 1.984π . Read `bq.precision`, `bq.shortfalls` and the warnings. For each, say in one sentence which of the three requirements in §6 failed, and whether more nodes would help.

Exercise 6 (the whole pipeline). Take `L_shape()`, build an `MPSEigensolver` with `from_domain`, solve for the first eigenvalue, and check that `eigenfunction_coef` returns coefficients whose Gram is 1 to 10^{-13} . Then verify the same norm against interior cubature (`lappy.cubature.polygon_cubature`) and explain which of the two you would trust if they disagreed at 10^{-9} .

11 Answers to check against

Run on the code as of this writing. If you get these, you have wired things up correctly; if you get something else, the discrepancy is the interesting part.

Exercise	quantity	value
1	disk, rect: $ \sum w(r \cdot n) - 2 \Omega $	$\leq 3 \times 10^{-15}$, any x_0
1	ellipse (2,1), default	1.4×10^{-15} at 168 nodes
1	ellipse (2,1), <code>resolve_geometry=False</code>	1.5×10^{-5} at 46 nodes
2	disk J_2 mode, quadrature	0.181230398151804
2	$\frac{\pi}{2} J_3(j_{2,1})^2$	0.181230398151804
3	fitted rates, $\gamma = \frac{1}{3}, \frac{2}{3}, \frac{3}{2}$	2.64, 3.31, 4.96
3	predicted $2\gamma + 2$	2.67, 3.33, 5.00
4	octagon ($\nu = 4/3$), default	<code>cornerinterp</code> , 824 nodes
4	octagon, <code>nonintegral=False</code>	<code>legendre</code> , 96 nodes
4	hexagon ($\nu = 3/2$), either way	<code>legendre</code> , 84 nodes
5	<code>ellipse(6,1)</code>	<code>precision</code> = 9.9×10^{-5} , segment shortfall
5	sector of 1.984π	<code>precision</code> = ∞ , corner demoted
6	<code>L_shape</code> , boundary Gram	$ G - 1 = 2.2 \times 10^{-16}$
6	<code>L_shape</code> , interior cubature	$ \cdot - 1 = 5.9 \times 10^{-11}$

Exercise 3's fit breaks down at $\gamma = 4.5$ because the error has already reached roundoff ($\sim 10^{-15}$) by order 32; there is no trend left to fit. Exercise 4's hexagon is the control: $\nu = 3/2$ gives $\gamma = 1$, an exact polynomial, so nothing should change and nothing does. Exercise 5's two failures are deliberately different — one names a *segment* and blames the parametrization, the other names a *corner* and refuses to quote a number at all. Exercise 6 is the one to think hardest about: the boundary route is five orders better than the interior cubature it is being compared to, which is why the comparison is reported as a cross-check and not as an error bar.

Where to read next

- `docs/corner_quadrature.pdf` — the research record: the corner rule derived properly, the failure of the graded rule quantified, and an addendum listing what did not survive contact with real domains.
- `docs/rellich.md` — the identity in its general (Zaremba) form, covering mixed boundary conditions.
- `docs/eigfun_integrals.md` — the API and its accuracy record.
- `lappy/eigfun_integrals.py` — the implementation. The docstrings carry the measurements; read `boundary_quadrature` and `corner_specs` first.
- `tests/test_eigfun_integrals.py` — worked, checked examples of every claim above, against closed forms.